Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки

Кафедра обчислювальної техніки

Дискретна математика

Лабораторна робота №1

«Множини: основні властивості та операції над ними, діаграми Венна».

Виконав: студент
групи ІО-42
Куліков М. М.
Номер у списку групи: 18
Перевірив:
Пономаренко А. М.

Київ – 2024

Лабораторна робота №1

Тема: «Множини: основні властивості та операції над ними, діаграми Венна».

Мета: вивчити основні аксіоми, закони і теореми теорії множин, навчитися застосовувати їх на практиці. Обчислити логічний вираз шляхом послідовного застосування операцій над множинами.

Загальне завдання:

1. Повторити матеріал: «Бібліотека tkinter (віджети)» та виконати лабораторну роботу з застосуванням графічного інтерфейсу.
2. Спростити логічний вираз з застосуванням тотожностей алгебри множин.
3. В окремому модулі написати функцію обчислення початкового логічного виразу (1), вибраного відповідно до індивідуального варіанта.
4. В окремому модулі написати функцію обчислення спрощеного логічного виразу.
5. В окремому модулі написати функцію виконання логічної операції (2), вибраної відповідно до індивідуального варіанта.
6. В окремому модулі виконати порівняння результатів:
   А) обчислення початкового та спрощеного виразу
   Б) виконання логічної операції Вашою функцією та відповідною стандартною логічною операцією або функцією Python.

Теоретичні основи:

Властивості множин:

Множина – є сукупність визначених об'єктів, різних між собою, об’єднаних за певною ознакою чи властивістю.

Множини позначають **великими** латинськими буквами. Об'єкти, що складають множини, називають елементами і позначають **малими** буквами латинського алфавіту.

На практиці часто застосовують такі позначення:

N – множина натуральних чисел;
P – множина додатних цілих чисел;
Z – множина додатних і від’ємних цілих чисел, включаючи нуль;
Q – множина раціональних чисел;
R – множина дійсних чисел.

Якщо множина не містить жодного елемента, її називають порожньою і позначають Ø.

Скінченна множина – це така множина, кількість елементів якої може бути виражена скінченним числом, причому не важливо, чи можемо ми порахувати це число в даний момент.

Нескінченна множина – це така множина, що не є скінченною

Способи задавання множин:

- ***перерахуванням,*** тобто списком всіх елементів. Такий спосіб задавання прийнятний тільки при задаванні скінченних множин. Позначення списку – у фігурних дужках.
- ***процедурою***, що породжує і описує спосіб одержання елементів множини із уже отриманих елементів або з інших об'єктів.
- ***описом характеристичних властивостей***, які повинні мати елементи множини.

Підмножина. Множину A називають підмножиною (або *включенням*) множини B ($A \subseteq B$), якщо кожен елемент множини A є елементом множини B, тобто, якщо $x \in A$, то $x \in B$. Якщо $A \subseteq B$ й $A \neq B$, то A називають строгою підмножиною й позначають $A \subset B$.

Рівність множин. Дві множини рівні ($A = B$), якщо всі їхні елементи збігаються. Множини A і B рівні, якщо $A \subseteq B$ і $B \subseteq A$.

Потужність множини. Кількість елементів у скінченній множині A називають *потужністю* множини A і позначають |A|.

Універсальною множиною U є множина, що має таку властивість, що всі розглянуті множини є її підмножинами.

Варто розрізняти поняття належності елементів множини і включення! Так, наприклад, якщо множина $A = \{1,3,6,13\}$, то $3 \in A$, $6 \in A$, але $\{3, 6\} \notin A$, у той час як $\{3, 6\} \subseteq A$.

Булеан. Множину всіх підмножин, що складаються з елементів множини A, називають *булеаном* $P(A)$.

<u>Операції над множинами</u>

Об'єднання. *Об'єднанням* множин A і B називають множину, що складається із всіх тих елементів, які належать хоча б одній з множин A або B. Об'єднання множин A і B позначають $A \cup B$. Це визначення рівносильне наступному:

$A \cup B = \{x \mid x \in A \text{ або } x \in B\}$

Перетин. *Перетином* множин A і B називають множину, що складається із всіх тих елементів, які належать як множині A, так і множині B. Перетин множин A і B позначають $A \cap B$. Це визначення рівносильне наступному:

$A \cap B = \{x \mid x \in A \text{ і } x \in B\}$

Доповнення. *Доповненням* (або *абсолютним доповненням*) множини A називають множину, що складається із всіх елементів універсальної множини, які не належать A. Доповнення множини A позначають $\bar{A}$. Це визначення рівносильне наступному:

$\bar{A} = U - A = \{x \mid x \in U \text{ і } x \notin A\}$

Різниця. *Різницею* множин A й B (або відносним доповненням) називають множину, що складається із всіх елементів множини A, які не належать B. Різницю множин A і B позначають $A - B$ або $A \setminus B$. Це визначення рівносильне наступному: $A - B = \{x \mid x \in A \text{ і } x \notin B\}$

Симетрична різниця. *Симетричною різницею* множин A і B називають множину, що складається з об'єднання всіх елементів, що належать множині A і не містяться в B , і елементів, що належать множині B і не містяться в A. Симетричну різницю множин A і B позначають $A + B$ або $A \Delta B$. Це визначення рівносильне наступному: $A \Delta B = (A \setminus B) \cup (B \setminus A)$.

Види операцій. Операції, які виконують над однією множиною, називають *унарними*. Операції, які виконують над двома множинами, називають *бінарними*. Прикладом унарної операції є знаходження доповнення. Прикладами бінарних операцій є об'єднання, перетин, різниця, симетрична різниця.

Діаграми Венна

Для графічної ілюстрації операцій над множинами даної універсальної множини U використовують діаграми Венна. Діаграма Венна – це зображення множини у вигляді геометричної множини, наприклад, кола. При цьому універсальну множину зображують у вигляді прямокутника. На рис. 1 зображені діаграми Венна для розглянутих операцій над множинами.

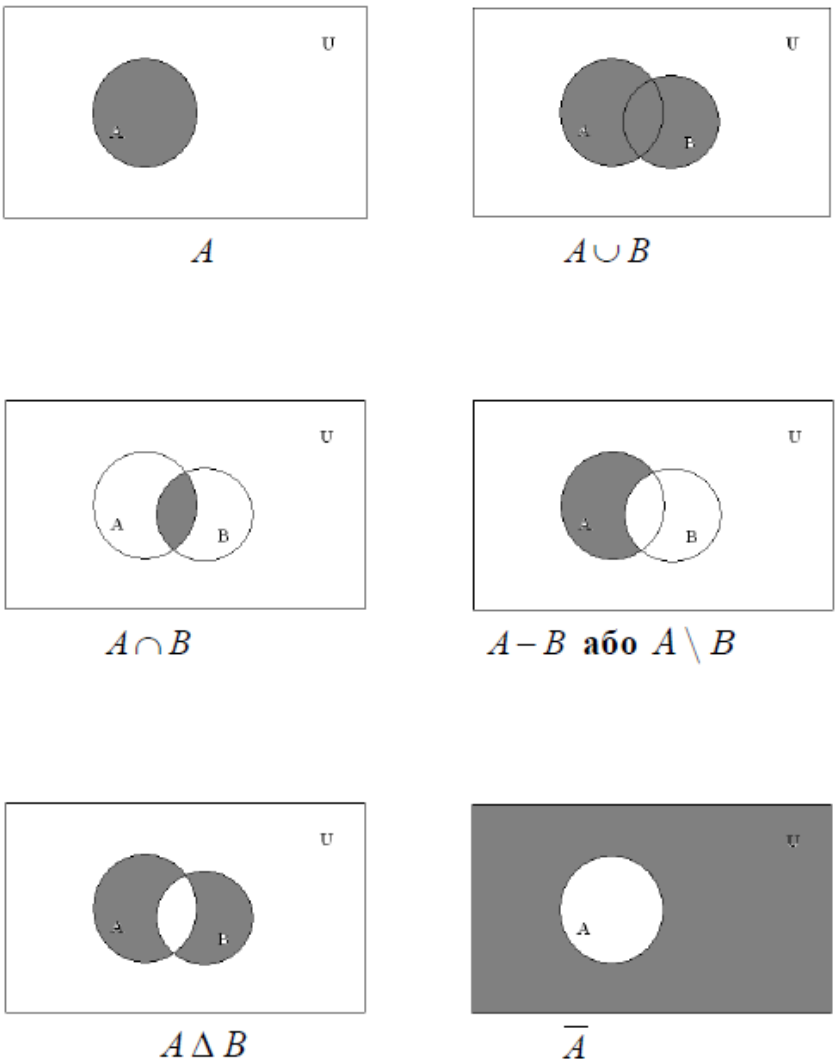

Рисунок 1 – діаграми Венна

Тотожності алгебри множин

1. Комутативність об'єднання $A \cup B = B \cup A$	1. Комутативність перетину $A \cap B = B \cap A$
2. Асоціативність об'єднання $A \cup (B \cup X) = (A \cup B) \cup X$	2. Асоціативність перетину $A \cap (B \cap X) = (A \cap B) \cap X$
3. Дистрибутивність об'єднання відносно перетину $A \cup (B \cap X) = (A \cup B) \cap (A \cup X)$	3. Дистрибутивність перетину відносно об'єднання $A \cap (B \cup X) = (A \cap B) \cup (A \cap X)$
4. Закони дій з пустою та універсальною множинами $A \cup \varnothing = A$ $A \cup \overline{A} = Y$ $A \cup Y = Y$	4. Закони дій з пустою та універсальною множинами $A \cap Y = A$ $A \cap \overline{A} = \varnothing$ $A \cap \varnothing = \varnothing$
5.Закон ідемпотентності об'єднання Термін **ідемпотентність** означує властивість математичного об'єкта, яка проявляється в тому, що повторна дія над об'єктом *не змінює* його $A \cup A = A$	5. Закон ідемпотентності перетину $A \cap A = A$
6. Закон де Моргана $\overline{A \cup B} = \overline{A} \cap \overline{B}$	6. Закон де Моргана $\overline{A \cap B} = \overline{A} \cup \overline{B}$
7. Закон поглинання $A \cup (A \cap B) = A$	7. Закон поглинання $A \cap (A \cup B) = A$
8. Закон склеювання $(A \cap B) \cup (A \cap \overline{B}) = A$	8. Закон склеювання $(A \cup B) \cap (A \cup \overline{B}) = A$
9. Закон Порецького $A \cup (\overline{A} \cap B) = A \cup B$	9. Закон Порецького $A \cap (\overline{A} \cup B) = A \cap B$
10. Закон подвійного доповнення $\overline{\overline{A}} = A$	
11. Визначення операції «різниця»: $A \setminus B = A \cap \overline{B}$.	
12. Визначення операції «симетрична різниця»: $A \Delta B = (A \setminus B) \cup (B \setminus A)$.	

Загальний порядок виконання роботи:

A. Вибрати номер Z індивідуального варіанта відповідно до виразу:

Z=(i+G%60)%30+1, де i – номер у списку групи, G – числова складова назви групи.

Програма обчислення варіанта:

```
G=64
N=1
print("Моя група: ІО -",G)
print("Мій номер у групі:",N)
print("Мій варіант:",(N+G%60)%30+1)
```

B. Максимально спростити логічний вираз. Для спрощення використати тотожності алгебри множин та визначення логічних операцій. Спрощення є максимальним, якщо формула містить тільки одне входження кожної множини.

C. Створити блок-схему послідовності обчислення початкового логічного виразу.

D. Створити блок-схему послідовності обчислення спрощеного логічного виразу.

E. З використанням блок-схем створити проект, який містить модуль з функцією для обчислення початкового виразу (1) та модуль з функцією для обчислення спрощеного виразу.

F. З використанням блок-схем, заданих у лабораторній роботі, створити модуль з функцією для виконання логічної операції (2), вибраної відповідно до варіанта.

G. В основному файлі виконати перевірку правильності спрощення виразу з виводом відповідного повідомлення.

H. Як елементи множин можуть бути використані числа від 0 до 255.

I. Лабораторну роботу виконувати з застосуванням мови Python та бібліотеки tkinter.

<u>Вимоги до інтерфейсу:</u>

Програма повинна складатися з 5-ти вікон.

1) **Вміст вікна №1.**
   1. Головне меню, яке повинно включати виклик вікна №2, вікна №3, вікна №4 та вікна №5.
   2. Віджети виводу П.І.Б студента, номера групи, номера у списку та віджет виводу результатів обчислення варіанту відповідно да програми, що задана у завданні – пункт (А) загального порядку виконання лабораторної роботи).
   3. Віджети для задавання потужності множин A, B і C.
   4. Віджети для формування випадковим чином множин A,B і C заданої потужності.
   5. Віджети, що дають можливість ручного вводу множин A,B і C.
   6. Віджет для задавання діапазону цілих чисел, які будемо вважати універсальною множиною.
2) **Вміст вікна №2**
   1. Віджети для відображення елементів множин A, B і C.
   2. Віджети запуску покрокового виконання початкового виразу (1). Одним кроком вважати виконання однієї логічної операції.
   3. Віджети відображення множин-операндів та множини-результату кожної логічної операції.
   4. Віджет відображення множини D та віджет для виконання команди збереження даного результату у файлі.
3) **Вміст вікна №3**
   1. Віджети для відображення елементів множин A, B і C.
   2. Віджети запуску покрокового виконання спрощеного логічного виразу. Одним кроком вважати виконання однієї логічної операції.
   3. Віджети відображення множин-операндів та множини-результату кожної логічної операції.
   4. Віджет відображення множини D, та віджет для виконання команди збереження даного результату у файлі.
4) **Вміст вікна №4**
   1. Віджети для відображення елементів множин X і Y.

2. Віджет відображення результату Z виконання заданої у індивідуальному варіанті логічної операції (2) над множинами X і Y, яка виконана за допомогою написаної вами функції.

5) **Вміст вікна №5**

1. Віджети для зчитування з файлу та відображення множини, яка є результатом обчислення початкового виразу.
2. Віджети для зчитування з файлу та відображення множини, яка є результатом обчислення спрощеного виразу.
3. Віджети для зчитування з файлу та відображення множини Z, яка є результатом обчислення операції над множинами X і Y, яка виконана за допомогою написаної вами функції.
4. Віджети для запуску та відображення логічної операції над множинами X і Y, яка є результатом обчислення Z з використанням стандартної функції Python.
5. Віджети для порівняння результатів виводу у пунктах 1 та 2 та у пунктах 3 та 4.

Індивідуальне завдання:

Завдання виконане згідно 18 варіанту.

№ вар		Вираз для обчислення
1	(1) (2)	$D = ((A \setminus B) \cup (B \cap A)) \setminus (C \cup B)$ $X = A; Y = B; Z = X \setminus Y$

Розрахунок варіанту:

Код програми:

```
G=42
N=18
print("Моя група: ІО -",G)
print("Мій номер у групі:",N)
print("Мій варіант:",(N+G%60)%30+1)
```

Результат виконання програми:

```
"D:\Documents\Education\KPI\2 Term\D
Моя група: ІО - 42
Мій номер у групі: 18
Мій варіант: 1
```

Рисунок 1 – результат обрахунку варіанта

Спрощений вираз:

1. $D = ((A \setminus B) \cup (B \cap A)) \setminus (C \cup B)$ (1.1)

Спростимо першу частину виразу:

$((A \setminus B) \cup (B \cap A)) =$ Спростимо різницю

$= ((A \cap \overline{B}) \cup (B \cap A)) =$ Вик. властивості комутативн.

$= ((A \cap \overline{B}) \cup (A \cap B)) =$ За законом склеювання

$= A$

Тепер вираз D має такий вигляд:

$D = A \setminus (C \cup B)$ (1.2)

Вираз є спрощеним, адже він містить тільки одне входження кожної множини.

Також, застосуючи спрощення різниці, маємо:

$D = A \setminus (C \cup B) =$ Спростимо різницю

$= A \cap \overline{(C \cup B)} =$ Вик. закон де Моргана

$= A \cap (\overline{C} \cap \overline{B}) =$ оскільки вик. перетин

$= A \cap \overline{C} \cap \overline{B}$ (1.3)

Рисунок 2 - спрощення виразу D

Блок схеми послідовності обчислення логічних виразів:

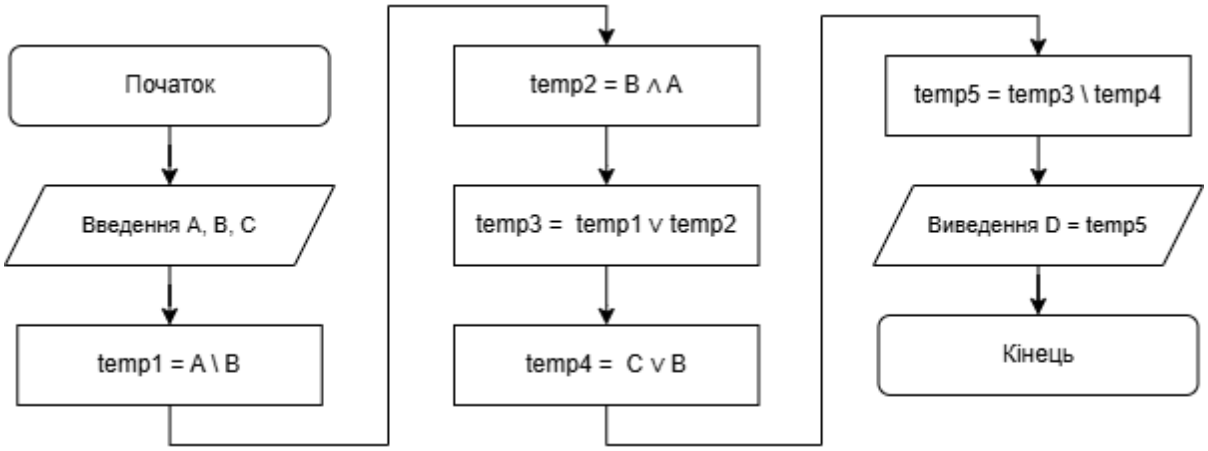

Рисунок 3 - блок-схема послідовності обчислення початкового логічного виразу

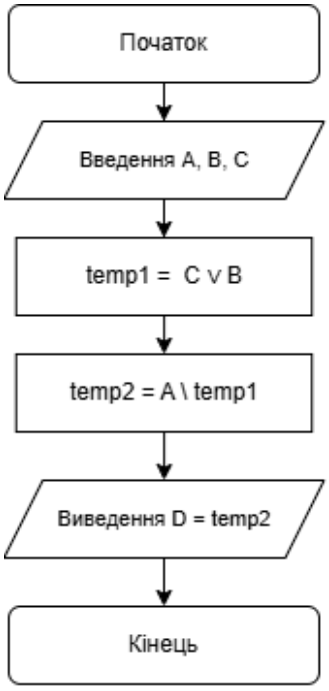

Рисунок 4 - блок-схема послідовності обчислення спрощеного логічного виразу

Код програми:

Вміст файлу functions.py:

```
 #для розрахунку варіанту
def calculate_var(group_fullnum, personal_num):
    return (group_fullnum+personal_num%60)%30+1

#деякі стандартні функції python, названі відповідними назвами
def union_python(a, b):
    return a | b

def intersection_python(a, b):
    return a & b

def difference_python(a, b):
    return a - b

#Для початкової функції
def calc_step1(a, b):
    return difference_python(a, b)
```

```
def calc_step2(a, b):
    return intersection_python(b, a)

def calc_step3(step1, step2):
    return union_python(step1, step2)

def calc_step4(c, b):
    return union_python(c, b)

def calc_step5(step3, step4):
    return difference_python(step3, step4)

def calculate_default_expression(a, b, c):
    step1 = calc_step1(a, b)
    yield step1
    step2 = calc_step2(a, b)
    yield step2
    step3 = calc_step3(step1, step2)
    yield step3
    step4 = calc_step4(c, b)
    yield step4
    yield calc_step5(step3, step4)

def calc_simplified_step1(c, b):
    return union_python(c, b)

def calc_simplified_step2(a, toDif):
    return difference_python(a, toDif)

def calculate_simplified_expression(a, b, c):
    step1 = calc_simplified_step1(c, b)
    yield step1
    yield calc_simplified_step2(a, step1)

# для вікна 4
def difference(a, b):
    result = set()
    for element in a:
        if element not in b:
```

```
            result.add(element)
    return result

def calculate_z(x, y):
    return difference(x, y)

#для вікна 5
def calculate_pythonized_z(x, y):
    return difference_python(x, y)
```

Вміст файлу LR1.py:

```
import functions
import tkinter as tk
from tkinter import messagebox
from tkinter import filedialog
import random

group = 42
group_num = 18

class Window1:
    def __init__(self, root):
        self.root = root
        self.root.title("Вікно #1")
        self.set_a = set()
        self.set_b = set()
        self.set_c = set()
        self.universal_set = set()
        menu = tk.Menu(self.root)
        self.root.config(menu=menu)
        menu.add_command(label="Вікно #2", command=self.open_window2)
        menu.add_command(label="Вікно #3", command=self.open_window3)
        menu.add_command(label="Вікно #4", command=self.open_window4)
        menu.add_command(label="Вікно #5", command=self.open_window5)
        frame_info = tk.Frame(self.root)
        frame_info.pack(pady=10)
        tk.Label(frame_info, text="ПІБ студента: Куліков Максим Миколайович").grid(row=0,
column=0)
        tk.Label(frame_info, text=f"Група: ІО-{group}").grid(row=1, column=0)
        tk.Label(frame_info, text=f"Номер у списку: {group_num}").grid(row=2, column=0)
```

```
        tk.Label(frame_info, text=f"Розрахований варіант: {functions.calculate_var(group,
group_num)}").grid(row=3, column=0)
        self.mode = tk.StringVar(value="auto")
        frame_mode = tk.Frame(self.root)
        frame_mode.pack(pady=10)
        tk.Radiobutton(frame_mode, text="Автоматичне генерування", variable=self.mode,
value="auto", command=self.toggle_mode).grid(row=0, column=0, padx=10)
        tk.Radiobutton(frame_mode, text="Ручне введення", variable=self.mode, value="manual",
command=self.toggle_mode).grid(row=0, column=1, padx=10)
        self.frame_inputs = tk.Frame(self.root)
        self.frame_inputs.pack(pady=10)
        self.frame_manual_inputs = tk.Frame(self.frame_inputs)
        self.frame_manual_inputs.grid(row=0, column=0, sticky="nsew")
        self.frame_auto_inputs = tk.Frame(self.frame_inputs)
        self.frame_auto_inputs.grid(row=0, column=0, sticky="nsew")
        self.create_auto_inputs()
        self.create_manual_inputs()
        self.toggle_mode()
        frame_results = tk.Frame(self.root)
        frame_results.pack(pady=10)
        self.label_A = tk.Label(frame_results, text="A: {}")
        self.label_A.pack()
        self.label_B = tk.Label(frame_results, text="B: {}")
        self.label_B.pack()
        self.label_C = tk.Label(frame_results, text="C: {}")
        self.label_C.pack()
        self.label_U = tk.Label(frame_results, text="U: {}")
        self.label_U.pack()

    def toggle_mode(self):
        if self.mode.get() == "manual":
            self.frame_manual_inputs.tkraise()
        else:
            self.frame_auto_inputs.tkraise()

    def create_manual_inputs(self):
        frame = self.frame_manual_inputs
        frame.grid_columnconfigure((0, 1), weight=1)
        tk.Label(frame, text="Елементи множини A (через кому):").grid(row=0, column=0)
        self.entry_manual_a = tk.Entry(frame, width=20)
        self.entry_manual_a.grid(row=0, column=1)
```

```
        tk.Label(frame, text="Елементи множини B (через кому):").grid(row=1, column=0)
        self.entry_manual_b = tk.Entry(frame, width=20)
        self.entry_manual_b.grid(row=1, column=1)
        tk.Label(frame, text="Елементи множини C (через кому):").grid(row=2, column=0)
        self.entry_manual_c = tk.Entry(frame, width=20)
        self.entry_manual_c.grid(row=2, column=1)
        tk.Label(frame, text="Універсум (через кому):").grid(row=3, column=0)
        self.entry_manual_u = tk.Entry(frame, width=20)
        self.entry_manual_u.grid(row=3, column=1)
        tk.Button(frame, text="Застосувати", command=self.generate_manual_sets).grid(row=4,
column=0, columnspan=2, pady=10)

    def create_auto_inputs(self):
        frame = self.frame_auto_inputs
        frame.grid_columnconfigure((0, 1, 2, 3), weight=1)
        tk.Label(frame, text="Кількість елементів A:").grid(row=0, column=0)
        self.entry_a = tk.Entry(frame, width=5)
        self.entry_a.grid(row=0, column=1)
        self.entry_a.insert(0, "4")
        tk.Label(frame, text="Кількість елементів B:").grid(row=1, column=0)
        self.entry_b = tk.Entry(frame, width=5)
        self.entry_b.grid(row=1, column=1)
        self.entry_b.insert(0, "4")
        tk.Label(frame, text="Кількість елементів C:").grid(row=2, column=0)
        self.entry_c = tk.Entry(frame, width=5)
        self.entry_c.grid(row=2, column=1)
        self.entry_c.insert(0, "4")
        tk.Label(frame, text="Діапазон чисел:").grid(row=3, column=0, pady=10)
        self.entry_range_min = tk.Entry(frame, width=10)
        self.entry_range_min.grid(row=3, column=1)
        self.entry_range_min.insert(0, "0")
        self.entry_range_max = tk.Entry(frame, width=10)
        self.entry_range_max.grid(row=3, column=2)
        self.entry_range_max.insert(0, "10")
        tk.Button(frame, text="Згенерувати", command=self.generate_auto_sets).grid(row=4,
column=0, columnspan=3, pady=10)

    def generate_manual_sets(self):
        def parse_input(entry):
```

```
            return {int(x) for x in entry.get().split(',') if x.strip().isdigit()}
        try:
            self.set_a = parse_input(self.entry_manual_a)
            self.set_b = parse_input(self.entry_manual_b)
            self.set_c = parse_input(self.entry_manual_c)
            if self.entry_manual_u.get().strip():
                manual_u = parse_input(self.entry_manual_u)
                self.universal_set = self.set_a | self.set_b | self.set_c | manual_u
            else:
                self.universal_set = set(self.set_a | self.set_b | self.set_c)
            self.update_results()
        except ValueError:
            messagebox.showerror("Помилка", "Введіть коректні значення для множин A, B, C.")

    def generate_auto_sets(self):
        def safe_sample(population, k):
            return set(random.sample(population, min(len(population), max(0, k))))

        try:
            min_val = int(self.entry_range_min.get().strip())
            max_val = int(self.entry_range_max.get().strip())
            count_a = int(self.entry_a.get().strip())
            count_b = int(self.entry_b.get().strip())
            count_c = int(self.entry_c.get().strip())
        except ValueError:
            messagebox.showerror("Помилка", "Введіть коректні числові значення!")
            return

        if min_val >= max_val:
            messagebox.showerror("Помилка", "Мінімальне значення має бути меншим за
максимальне!")
            return
        self.universal_set = set(range(min_val, max_val + 1))
        max_elements = len(self.universal_set)
        if count_a > max_elements or count_b > max_elements or count_c > max_elements:
            messagebox.showwarning("Попередження", f"Розмір множини не може перевищувати
{max_elements} елементів!")
        self.set_a = safe_sample(list(self.universal_set), count_a)
        self.set_b = safe_sample(list(self.universal_set), count_b)
```

```
        self.set_c = safe_sample(list(self.universal_set), count_c)
        self.update_results()

    def update_results(self):
        def format_set(s):
            return "{}" if not s else str(s)
        self.label_A.config(text=f"A: {format_set(self.set_a)}")
        self.label_B.config(text=f"B: {format_set(self.set_b)}")
        self.label_C.config(text=f"C: {format_set(self.set_c)}")
        self.label_U.config(text=f"U: {format_set(self.universal_set)}")

    def open_window2(self):
        if not self.are_sets_created():
            messagebox.showinfo("Попередження", "Не усі множини створені, але ви можете
переглянути це вікно.")
        Window2(self.root, self.set_a, self.set_b, self.set_c)

    def open_window3(self):
        if not self.are_sets_created():
            messagebox.showinfo("Попередження", "Не усі множини створені, але ви можете
переглянути це вікно.")
        Window3(self.root, self.set_a, self.set_b, self.set_c)

    def open_window4(self):
        if not self.are_sets_created():
            messagebox.showinfo("Попередження", "Не усі множини створені, але ви можете
переглянути це вікно.")
        Window4(self.root, self.set_a, self.set_b)

    def open_window5(self):
        if not self.are_sets_created():
            messagebox.showinfo("Попередження", "Не усі множини створені, але ви можете
переглянути це вікно.")
        Window5(self.root, self.set_a, self.set_b)

    def are_sets_created(self):
        return bool(self.set_a) or bool(self.set_b) or bool(self.set_c)

class Window2:
```

```
    def __init__(self, root, set_a, set_b, set_c):
        self.root = root
        self.set_a = set_a
        self.set_b = set_b
        self.set_c = set_c
        self.step = 0
        self.result_d = set()
        self.calculator = functions.calculate_default_expression(self.set_a, self.set_b, self.set_c)
        self.window = tk.Toplevel(self.root)
        self.window.title("Вікно #2")
        self.label_D = tk.Label(self.window, text="D: {}")
        self.label_D.pack(pady=10)
        self.label_step1 = tk.Label(self.window, text="Крок 1: A \\ B")
        self.label_step1.pack(anchor="center")
        self.label_step1_res = tk.Label(self.window, text="Результат буде тут", fg="blue")
        self.label_step1_res.pack(anchor="center")
        self.label_step2 = tk.Label(self.window, text="Крок 2: B ∩ A")
        self.label_step2.pack(anchor="center")
        self.label_step2_res = tk.Label(self.window, text="Результат буде тут", fg="blue")
        self.label_step2_res.pack(anchor="center")
        self.label_step3 = tk.Label(self.window, text="Крок 3: (A \\ B) ∪ (B ∩ A)")
        self.label_step3.pack(anchor="center")
        self.label_step3_res = tk.Label(self.window, text="Результат буде тут", fg="blue")
        self.label_step3_res.pack(anchor="center")
        self.label_step4 = tk.Label(self.window, text="Крок 4: C ∪ B")
        self.label_step4.pack(anchor="center")
        self.label_step4_res = tk.Label(self.window, text="Результат буде тут", fg="blue")
        self.label_step4_res.pack(anchor="center")
        self.label_step5 = tk.Label(self.window, text="Крок 5: ((A \\ B) ∪ (B ∩ A)) \\ (C ∪ B)")
        self.label_step5.pack(anchor="center")
        self.label_step5_res = tk.Label(self.window, text="Результат буде тут", fg="blue")
        self.label_step5_res.pack(anchor="center")
        self.calculate_button = tk.Button(self.window, text="Обчислити",
command=self.calculate_step_by_step)
        self.calculate_button.pack(pady=10)
        self.save_button = tk.Button(self.window, text="Зберегти результат",
command=self.save_result)
        self.save_button.pack(pady=10)
```

```
    def calculate_step_by_step(self):
        try:
            step_result = next(self.calculator)
            if not step_result:
                step_result = "{}"
            if self.step == 0:
                self.label_step1_res.config(text=f"{step_result}")
            elif self.step == 1:
                self.label_step2_res.config(text=f"{step_result}")
            elif self.step == 2:
                self.label_step3_res.config(text=f"{step_result}")
            elif self.step == 3:
                self.label_step4_res.config(text=f"{step_result}")
            elif self.step == 4:
                self.label_step5_res.config(text=f"{step_result}")
                self.result_d = step_result
                self.label_D.config(text=f"D: {self.result_d}")
                self.calculate_button.config(state=tk.DISABLED)
            self.step += 1
        except StopIteration:
            self.calculate_button.config(state=tk.DISABLED)

    def save_result(self):
        file_path = filedialog.asksaveasfilename(defaultextension=".txt", filetypes=[("Text files",
"*.txt")])
        if file_path:
            with open(file_path, "w") as file:
                file.write(f"Результат обчислення D: {self.result_d}")

class Window3:
    def __init__(self, root, set_a, set_b, set_c):
        self.root = root
        self.set_a = set_a
        self.set_b = set_b
        self.set_c = set_c
        self.step = 0
        self.result_d = set()
        self.calculator = functions.calculate_simplified_expression(self.set_a, self.set_b, self.set_c)
        self.window = tk.Toplevel(self.root)
```

```
        self.window.title("Вікно #3")
        self.label_D = tk.Label(self.window, text="D: {}")
        self.label_D.pack(pady=10)
        self.label_step1 = tk.Label(self.window, text="Крок 1: C ∪ B")
        self.label_step1.pack(anchor="center")
        self.label_step1_res = tk.Label(self.window, text="Результат буде тут", fg="blue")
        self.label_step1_res.pack(anchor="center")
        self.label_step2 = tk.Label(self.window, text="Крок 2: A \\ (C ∪ B) = ")
        self.label_step2.pack(anchor="center")
        self.label_step2_res = tk.Label(self.window, text="Результат буде тут", fg="blue")
        self.label_step2_res.pack(anchor="center")
        self.calculate_button = tk.Button(self.window, text="Обчислити",
command=self.calculate_step_by_step)
        self.calculate_button.pack(pady=10)
        self.save_button = tk.Button(self.window, text="Зберегти результат",
command=self.save_result)
        self.save_button.pack(pady=10)

    def calculate_step_by_step(self):
        try:
            step_result = next(self.calculator)
            if not step_result:
                step_result = "{}"
            if self.step == 0:
                self.label_step1_res.config(text=f"{step_result}")
            elif self.step == 1:
                self.label_step2_res.config(text=f"{step_result}")
                self.result_d = step_result
                self.label_D.config(text=f"D: {self.result_d}")
                self.calculate_button.config(state=tk.DISABLED)
            self.step += 1
        except StopIteration:
            self.calculate_button.config(state=tk.DISABLED)

    def save_result(self):
        file_path = filedialog.asksaveasfilename(defaultextension=".txt", filetypes=[("Text files",
"*.txt")])
        if file_path:
            with open(file_path, "w") as file:
```

```
            file.write(f"Результат спрощеного обчислення D: {self.result_d}")


class Window4:
    def __init__(self, root, set_x, set_y):
        self.root = root
        self.set_x = set_x
        self.set_y = set_y
        self.result_z = set()
        self.window = tk.Toplevel(self.root)
        self.window.title("Вікно #4")
        self.label_X = tk.Label(self.window, text=f"X = {self.set_x}")
        self.label_X.pack()
        self.label_Y = tk.Label(self.window, text=f"Y = {self.set_y}")
        self.label_Y.pack()
        self.label_Z = tk.Label(self.window, text="X\\Y = Z: ")
        self.label_Z.pack()
        self.calculate_button = tk.Button(self.window, text="Обчислити Z", command=self.calculate_z)
        self.calculate_button.pack()
        self.save_z_button = tk.Button(self.window, text ="Зберегти Z до файлу",
command=self.save_z)
        self.save_z_button.pack()
    def calculate_z(self):
        self.result_z = functions.calculate_z(self.set_x, self.set_y)
        self.label_Z.config(text=f"X\\Y = Z: {self.result_z if self.result_z else '{}'}")
    def save_z(self):
        if self.result_z:
            file_path = filedialog.asksaveasfilename(defaultextension=".txt", filetypes=[("Text files",
"*.txt")])
            if file_path:
                with open(file_path, "w") as file:
                    file.write(f"Результат обчислення Z: {self.result_z}")
        else:
            messagebox.showerror("Помилка", "Спочатку обчисліть Z!")



class Window5:
        def __init__(self, root, x_set, y_set):
            self.root = root
            self.x_set = x_set
```

```
        self.y_set = y_set
        self.window = tk.Toplevel(self.root)
        self.window.title("Вікно #5")
        self.window.minsize(width=200, height=250)
        self.label_expr = tk.Label(self.window, text="Початковий вираз: Невідомо")
        self.label_expr.pack()
        self.label_simplified_expr = tk.Label(self.window, text="Спрощений вираз: Невідомо")
        self.label_simplified_expr.pack()
        self.label_z = tk.Label(self.window, text="Z: Невідомо")
        self.label_z.pack()
        self.label_calc_z = tk.Label(self.window, text="Обчислений Z: Невідомо")
        self.label_calc_z.pack()
        tk.Button(self.window, text="Завантажити множину D", command=self.load_d_set).pack()
        tk.Button(self.window, text="Завантажити спрощену множину D",
command=self.load_simplified_d_set).pack()
        tk.Button(self.window, text="Завантажити множину Z", command=self.load_z_set).pack()
        tk.Button(self.window, text="Обчислити множину Z", command=self.calculate_z).pack()
        tk.Button(self.window, text="Порівняти результати", command=self.compare_results).pack()
        self.label_comparison = tk.Label(self.window, text="Результат порівняння: Невідомо")
        self.label_comparison.pack()
    def load_d_set(self):
        self.d_set = self.load_set_from_file("Початковий вираз", self.label_expr)

    def load_simplified_d_set(self):
        self.simplified_d_set = self.load_set_from_file("Спрощений вираз",
self.label_simplified_expr)

    def load_set_from_file(self, label_text, label):
        file_path = filedialog.askopenfilename(filetypes=[("Текстові файли", "*.txt"), ("Усі файли",
"*.*")])
        if file_path:
            try:
                with open(file_path, "r") as file:
                    content = file.read().strip()
                    start_index = content.find("{")
                    end_index = content.find("}", start_index)
                    if start_index != -1 and end_index != -1:
                        numbers = content[start_index + 1:end_index].strip()
                        if numbers:
                            set_data = set(map(int, filter(None, numbers.split(','))))
                        else:
```

```
                set_data = set()
            else:
                set_data = set()
            label.config(text=f"{label_text}: {set_data if set_data else '{}'}")
            return set_data
        except Exception as e:
            messagebox.showerror("Помилка", f"Не вдалося завантажити {label_text}: {e}")
            return None

    def load_z_set(self):
        self.z_set = self.load_set_from_file("Z", self.label_z)

    def calculate_z(self):
        if hasattr(self, 'x_set') and hasattr(self, 'y_set'):
            self.calc_z = functions.calculate_pythonized_z(self.x_set, self.y_set)
            self.label_calc_z.config(text=f"Обчислений Z: {self.calc_z if self.calc_z else '{}'}")
        else:
            messagebox.showerror("Помилка", "Створіть множини X і Y (A, B)!")

    def compare_results(self):
        comparison_results = []
        if hasattr(self, 'd_set') and hasattr(self, 'simplified_d_set'):
            comparison_results.append(
                "D і спрощена D співпадають" if self.d_set == self.simplified_d_set else "D і спрощена
D не співпадають")
        else:
            comparison_results.append("Завантажте множини D!")
        if hasattr(self, 'calc_z') and hasattr(self, 'z_set'):
            comparison_results.append(
                "Обчислений Z і завантажений Z співпадають" if self.calc_z == self.z_set else
"Обчислений Z і завантажений Z не співпадають")
        else:
            comparison_results.append("Спочатку завантажте та обчисліть Z!")
        self.label_comparison.config(text="\n".join(comparison_results))

if __name__ == "__main__":
    root = tk.Tk()
    app = Window1(root)
    root.mainloop()
```

Результат виконання програми:

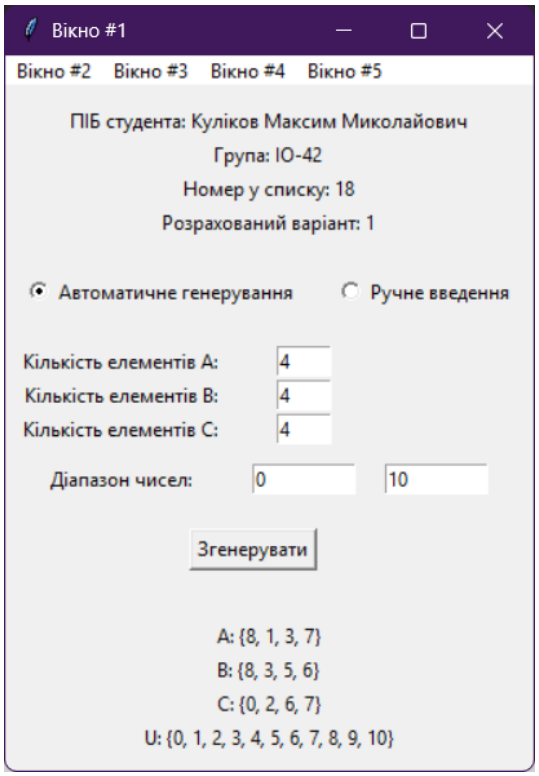

Рисунок 5 – результат виконання програми у вікні 1

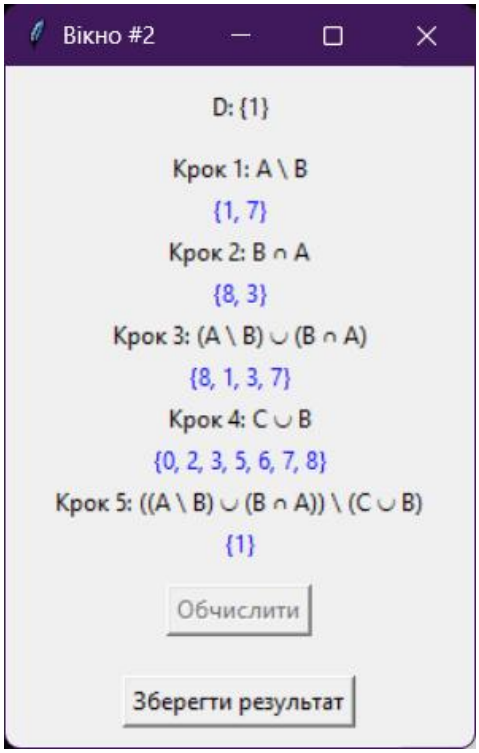

Рисунок 6 – результат виконання програми у вікні 2

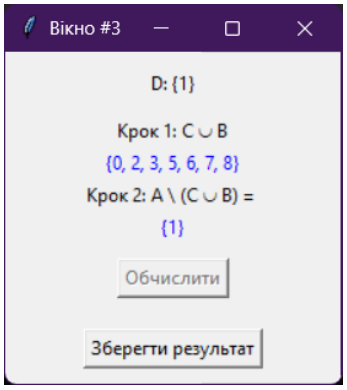

Рисунок 7 – результат виконання програми у вікні 3

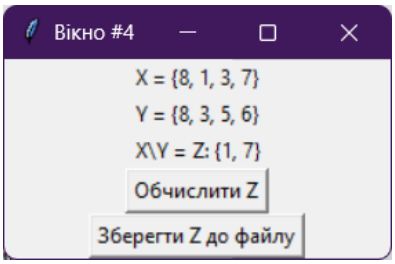

Рисунок 8 – результат виконання програми у вікні 4

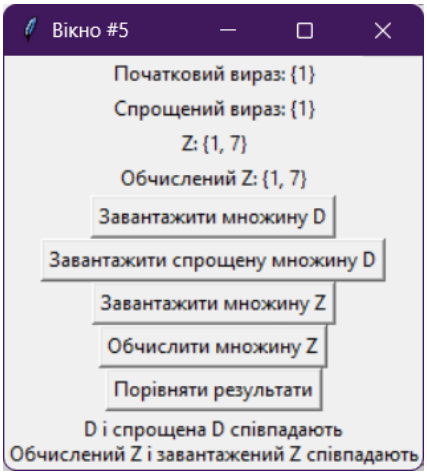

Рисунок 9 – результат виконання програми у вікні 5

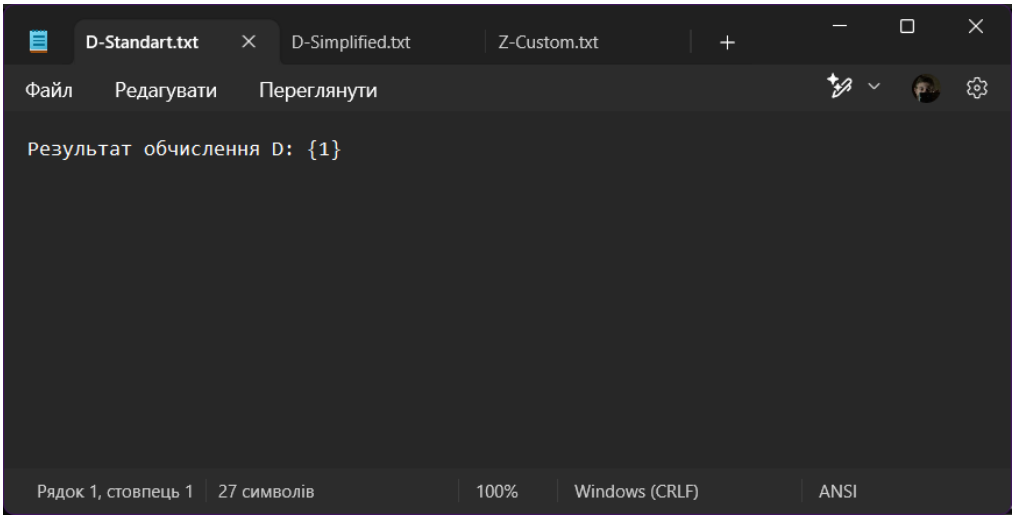

Рисунок 10 – результат збереження множини D у файл

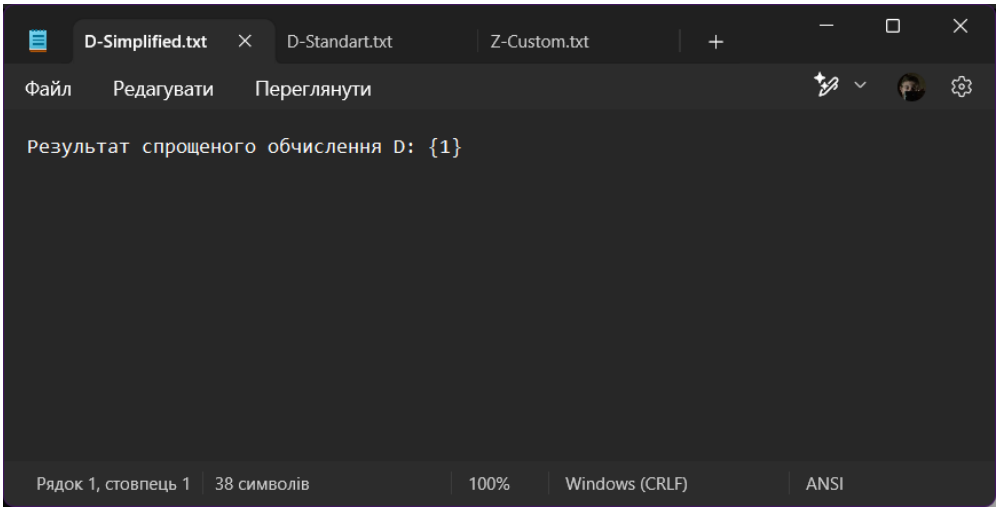

Рисунок 11 – результат збереження множини D спрощеним чином у файл

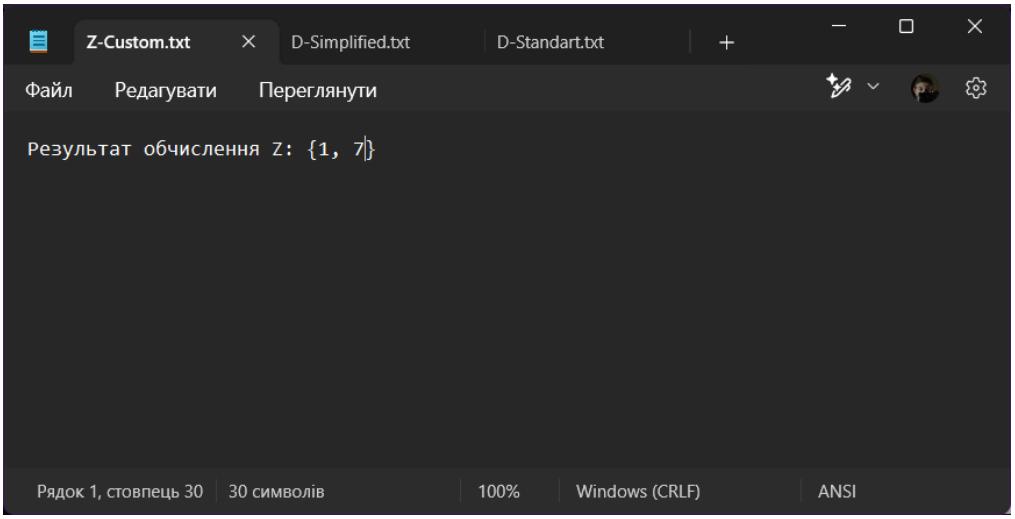

Рисунок 12 – результат збереження множини Z за допомогою власної функції у файл

Висновки:

Отже, під час виконання лабораторної роботи я вивчив основні аксіоми, закони і теореми теорії множин, а також навчився застосовувати їх на практиці. Для реалізації операцій над множинами я використав мову програмування Python та бібліотеку Tkinter, що дозволило створити графічний інтерфейс для введення множин і виконання над ними логічних операцій. У результаті я не лише закріпив теоретичні знання, а й набув практичного досвіду роботи з Python та розробки GUI-додатків. У цілому, проблем не виникло, обидва методи працюють ідентично, однак для зручності читання коду краще використовувати вбудовані функції.